OPERATING SYSTEMS (UE25CS242A)

Unit 1 – Introduction and Process Management

Question Bank

Lecture 1 – Introduction

1. How do operating system objectives change when shifting from batch systems to interactive systems?
2. You are designing an Operating System for a supercomputer used exclusively for weather forecasting and another for a home entertainment tablet. Contrast the Operating System Goals for these two systems. How does the definition of efficiency differ between them?
3. Describe how the OS acts as an intermediary between application programs and computer hardware.
4. List the four components of a computer system and state the role of each.
5. What is the function of the interrupt vector in a computer system?
6. What is the role of a bootstrap program, what does it do, and where is it stored?
7. "Main memory can be viewed as a cache for secondary storage." Explain this statement. If the CPU needs data that is currently on the Hard Disk, describe the chain of copying required to get that data into the CPU Registers.
8. "A user program cannot directly access hardware." If a text editor needs to save a file to the disk, explain the mechanism it uses to bypass this restriction securely. What role does the Dual-Mode architecture play here?
9. If the interrupt vector gets corrupted during system operation, how would the OS respond to incoming hardware interrupts and what would be the visible impact?
10. Why is the Operating System described as Interrupt Driven? What would happen to the system's responsiveness if interrupts were disabled entirely?
11. During the boot process, the bootstrap program is stored in firmware rather than on the hard disk. What risk would arise if the bootstrap were stored on the hard disk instead?
12. Two processes simultaneously issue I/O requests, one uses interrupt-driven I/O and the other uses DMA. The system has a single memory bus. Analyze how memory bus contention could degrade the performance of the interrupt-driven process.
13. Briefly explain the difference between a Uniprocessor system and a Multiprocessor system. Why might a Multiprocessor system be preferred for a server handling thousands of transactions?
14. A computer system relies entirely on polling instead of interrupts for all I/O operations. Analyze the complete impact on CPU utilization, I/O performance, and system scalability.

Lecture 2 – OS Structure and Operations

15. Explain why protection mechanisms are essential in a multi-user operating system.
16. How does time-sharing improve CPU utilization in a multiprogrammed environment?
17. Discuss the role of hardware timers in preventing CPU monopolization.
18. What is dual-mode operation and why is it needed?
19. What is the difference between a trap and an interrupt?
20. What is the difference between Asymmetric Multiprocessing and Symmetric Multiprocessing?
21. What is the difference between multiprogramming and multitasking?
22. A disk controller has its own microprocessor that maintains a disk queue and runs its own scheduling algorithm. If this special-purpose processor were removed and all disk scheduling were handled by the main CPU, what would be the impact under heavy I/O load?
23. In a Symmetric Cluster, two nodes try to write to the same student record on the shared Storage Area Network at the exact same instant. What mechanism prevents data corruption here? Why is a simple local semaphore not sufficient?
24. A system administrator sets the Timer interrupt to occur every 1 microsecond. What is the negative impact of this configuration on the system's Throughput?
25. What are the advantages of clustered systems over single-system architectures? How do asymmetric and symmetric clustering differ?
26. Modern CPUs often support more than just two modes. What is the purpose of the VMM (Virtual Machine Manager) Mode? How does it relate to Guest VMs?

Lecture 3 – OS Services and Process Concept

27. Explain how operating systems provide controlled access to hardware resources.
28. A headless server in a data center crashes due to a memory error. Since there is no screen to display an error message, which OS service is responsible for handling this, and how does the administrator eventually find out what happened?
29. A program needs to write a 10MB video file to the disk. Why can't the system pass this 10MB of data directly through the CPU registers during the system call? What method is used instead?
30. You have written a program main.c. Before it becomes a running process, it goes through several steps. Explain the roles of the Compiler, Linker, and Loader in this process.
31. What are the five states a process can be in during its lifetime?

32. Two processes need to exchange data. The OS provides Shared Memory and Message Passing. Which method would you choose for exchanging small amounts of status data between two computers across a network? Justify.
33. Differentiate between a program and a process with suitable justification.
34. Why must each process be uniquely identified by the operating system?
35. Explain how multitasking improves system responsiveness.
36. When the CPU switches from Process P0 to Process P1, it performs a State Save and State Restore. Where exactly is this state saved? List four specific items stored in this data structure.
37. What is the difference between shared memory and message passing as inter-process communication methods?
38. A system has only a short-term scheduler and no long-term scheduler. Over time, users keep launching new processes. Identify what resource will be exhausted first and why.
39. "Context Switching is considered pure overhead." Justify this statement. Why don't we switch processes every single microsecond to create perfect multitasking?
40. A system is running 10 processes. 9 of them are I/O Bound and 1 is CPU Bound. Describe the behavior of the Ready Queue. Is it likely to be full or empty most of the time?
41. If the hardware has no memory protection mechanism and a user program writes to another process's memory, how does this affect the OS services of error detection, protection, and accounting?
42. Describe the responsibilities of the long-term, short-term, and medium-term schedulers and how they work together to manage processes.
43. How does the Process Lifecycle in a mobile OS like Android differ from a traditional desktop OS regarding background processes? Why might the OS terminate a background process without the user explicitly closing it?
44. Trace the complete life cycle of a process from creation to termination using the process state diagram and identify what triggers each transition.
45. The OS provides both a ready queue and multiple device queues. If a process gets stuck permanently in a device queue because its I/O device stops responding, what happens to that process's PCB and will the short-term scheduler ever see it again?
46. You run the command ps -el and see a process with status Z (Zombie). Explain exactly what this process is. Is it using CPU? Is it using Memory? Why is it still in the table?
47. The OS uses three types of schedulers. If a system removes the medium-term scheduler entirely, what impact would this have on memory management under heavy load?

Lecture 5 – Process Creation and Termination, System Calls for Process Management

48. Why does process creation involve duplication of process attributes?
49. Explain how the operating system ensures fairness among competing I/O requests.
50. What problems can arise if processes are not properly synchronized during I/O operations?
51. You are optimizing a system where a process needs to create a child that immediately calls exec() to run a new program. Would you use fork() or vfork()? Justify your choice based on memory overhead and performance.
52. What is the difference between an orphan process and a zombie process?
53. What does the exec() system call do?
54. What is the difference between fork() and exec()?
55. What is cascading termination?
56. A server runs for months. You check the process table and see hundreds of Zombie entries. Explain what a Zombie process is in terms of resource usage. Why is it a problem if they accumulate?
57. After fork(), the child is described as a copy of the parent. If the child modifies a global variable, does the parent see the change?
58. A process calls exec() after fork(). The exec() call succeeds. What happens to the child's original memory sections and does the PID change?
59. Process IDs are reused after processes terminate, but UNIX systems deliberately delay this reuse. If PIDs were reused immediately, what specific problem would a monitoring tool face?
60. Explain the complete execution flow of a program using fork(), exec(), and wait() working together.
61. During I/O execution, the requesting process is typically blocked. Why does the OS block the process instead of letting it spin in a loop checking for data? How does this improve System Throughput?

Lecture 7 – CPU Scheduling- FCFS Scheduling, SJF Scheduling

62. Explain how scheduling policies affect system responsiveness and throughput.
63. Differentiate between preemptive and non-preemptive scheduling by examining the specific process state transitions that trigger a scheduling decision. Why is non-preemptive scheduling considered cooperative?
64. List the five scheduling criteria used to evaluate CPU scheduling algorithms.

65. The Dispatcher is a distinct component from the scheduler. Elucidate the specific responsibilities of the dispatcher and explain how dispatch latency impacts system performance.
66. What is the difference between preemptive and non-preemptive scheduling?
67. What is the convoy effect in FCFS scheduling?
68. An Operating System designer must balance various scheduling criteria. Compare Throughput with Turnaround Time, and discuss the specific optimization goals for a scheduler regarding Waiting Time and Response Time.
69. Explain the CPU-I/O burst cycle and how the distribution of CPU burst lengths affects the choice of scheduling algorithm.
70. The First-Come, First-Served (FCFS) algorithm is technically simple but can lead to the Convoy Effect. Analyze this phenomenon and explain why FCFS is generally considered unsuitable for time-sharing systems.
71. Shortest-Job-First (SJF) is often cited as the optimal scheduling algorithm. Explain the metric by which it is considered optimal, and discuss the primary implementation challenge that prevents its direct use.
72. Consider processes P1, P2, P3 arriving at time 0 with burst times of 24ms, 3ms, and 3ms. Compare the Average Waiting Time if scheduled in different orders. What does this reveal about the sensitivity of FCFS?
73. If the dispatch latency is very high, how does this affect the scheduling criteria even if the scheduling algorithm itself makes optimal decisions?
74. Compare FCFS and SJF scheduling algorithms across all five scheduling criteria.
75. SJF requires knowing the next CPU burst length which is impossible. The OS uses exponential averaging to predict it. If the prediction is consistently wrong for a process whose burst pattern changes suddenly, how does this affect the scheduling decisions?
76. In a system using non-preemptive scheduling, a CPU-bound process with a very long burst is currently running and a high-priority I/O-bound process enters the ready queue. Analyze the cascading impact on both processes and the I/O device.
77. The ready queue can be organized as a FIFO queue, priority queue, tree, or unordered linked list. How does the choice of data structure affect the scheduler's performance for FCFS versus SJF?
78. SJF is optimal for minimum average waiting time but can cause starvation for long-burst processes. Analyze why this happens in both preemptive and non-preemptive SJF and what fundamental scheduling goal is violated.

79. A system designer must choose between FCFS, SJF, and SRTF for a web server handling both quick page requests and large file downloads. Evaluate each algorithm and recommend the best choice.

Lecture 8 – CPU Scheduling Algorithms (Part 2)

80. Priority Scheduling assigns the CPU based on rank rather than arrival time. Explain the convention used to define highest priority and discuss the critical problem of Starvation. How does Aging resolve this issue?
81. Conceptually, Shortest-Job-First (SJF) can be categorized as a specific type of Priority Scheduling. Analyze the relationship between these two algorithms. What determines the priority in an SJF context?
82. Round Robin (RR) scheduling is explicitly designed for time-sharing systems. Describe the mechanics of the RR algorithm, specifically focusing on the Ready Queue structure and the concept of the Time Quantum.
83. The performance of the Round Robin algorithm is highly sensitive to the size of the Time Quantum. Contrast the system behavior when the quantum is extremely large versus when it is extremely small.
84. The relationship between Time Quantum size and Average Turnaround Time is not linear. What is the general rule of thumb regarding CPU bursts and Time Quantum sizing to optimize performance?
85. In Round Robin, what is the maximum waiting time for a process before it gets the CPU again?
86. What is the difference between preemptive and non-preemptive priority scheduling?
87. What happens when the Round Robin time quantum is set extremely large?
88. A system uses Round Robin with a quantum of 20ms. An I/O-bound process uses only 2ms of its quantum. A CPU-bound process uses the full 20ms. Over 10 rounds, compare the total CPU time each type gets.
89. Compare Priority Scheduling and Round Robin Scheduling across all five scheduling criteria.
90. In Round Robin, if the time quantum is 1ms and context switch time is 0.5ms, calculate the CPU efficiency. At what point does reducing the quantum make the system slower?
91. A system uses priority scheduling with aging. A low-priority process eventually matches a high-priority real-time process. Analyze what the OS must do to handle this tie.
92. A company runs online banking, video rendering, background backups, and web browsing on a single server. Recommend a scheduling approach and justify how each workload would be handled.

Lecture 10– CPU Scheduling Algorithms (Part 3)

93. Multilevel Queue Scheduling introduces a rigid structure to the ready queue. Explain how this algorithm handles different classes of processes and discuss the two levels of scheduling decisions that must be made.
94. How does the Multilevel Feedback Queue (MLFQ) scheduler improve upon the standard Multilevel Queue approach? Detail the five parameters that define an MLFQ scheduler and explain how aging is implemented.
95. Consider a Multilevel Feedback Queue system with three queues: Q0 (RR, 8ms quantum), Q1 (RR, 16ms quantum), and Q2 (FCFS). Trace the path of a long CPU-bound process entering this system and explain how preemption is handled between queues.
96. In the context of Multiple-processor Scheduling, distinguish between Asymmetric Multiprocessing and Symmetric Multiprocessing (SMP). Why is SMP the standard for modern operating systems despite its higher complexity?
97. Explain the concept of Processor Affinity and its relationship to cache memory. How does this concept conflict with the goal of Load Balancing in an SMP system?
98. List the five parameters that define a Multilevel Feedback Queue scheduler.
99. How does the priority between queues work in the MLFQ example?
100. In Multilevel Queue scheduling, processes are permanently assigned to a queue. A process initially classified as batch later starts behaving like an interactive process. How does permanent queue assignment hurt this process?
101. In Multilevel Queue scheduling, CPU time is divided 80 percent to foreground and 20 percent to background. If the foreground queue becomes empty, what happens?
102. The MLFQ demotion method is too aggressive and there is no upgrade method. What happens to I/O-bound processes that occasionally have one long CPU burst?
103. In the MLFQ with Q0 (RR 8ms), Q1 (RR 16ms), Q2 (FCFS), analyze why this design inherently favors I/O-bound processes over CPU-bound processes.
104. A designer simplifies MLFQ to only two queues instead of three. What distinction between process types is lost and which processes suffer most?
105. Compare Multilevel Queue Scheduling and Multilevel Feedback Queue Scheduling in detail across queue assignment, flexibility, starvation handling, and adaptiveness.
106. A company runs video conferencing, online transactions, software compilation, and database backups. Design a suitable MLFQ scheduler specifying the number of queues, scheduling algorithm for each, quantum sizes, and upgrade and demotion rules.

Lecture 11 – Introduction to Shell and Cron

107. Analyze the operational flow of a Shell when a user enters a command. How does the shell distinguish between a Shell Variable and an Environment Variable in terms of scope and inheritance?

108. Explain the significance of the PATH environment variable. What specific problem does it solve for the user, and how does the shell utilize it during command execution?

109. What is cron and how does it work?

110. What is the difference between environment variables and shell variables?

111. Deconstruct the syntax of if statements and for loops in a Bash script. How do the control flow keywords break and continue alter the execution of a loop?

112. A user defines a shell variable in the parent shell and spawns a child shell. After exporting, the child can see it. If the child modifies the exported variable, does the parent see the change?

113. The shell searches for commands using PATH. If two programs with the same name exist in different PATH directories, which one gets executed and why?

114. Why is cron preferred over manually running scripts for repetitive system administration tasks?

115. A user wants to make an environment variable permanent so it is available every time they log in. How does Linux solve this problem?

116. How does the Linux shell process a command from the moment the user types it until the result is displayed?

117. A developer stores a database password as an exported environment variable. Analyze the security risk and why shell variables alone cannot solve the problem.

118. A malicious user adds a directory they control to the beginning of another user's PATH, placing a fake ls program there. Trace what happens when the victim types ls.

119. Compare shell variables and environment variables in detail across scope, inheritance, creation, persistence, use cases, and visibility.

120. You are tasked with scheduling a system backup script (backup.sh) to run automatically. Analyze the fields of a crontab entry and construct the specific line required to execute this script every Wednesday at 4:30 AM.